Apple Tree Reconstruction

Kieran Todd (34957731)
https://github.com/ktod0002/MMA3001-Project - https://ktod0002.github.io/MMA3001-Project/

The engineering problem

The use of autonomous robots for apple harvesting is becoming increasingly needed and prevalent. These robots should be able to safely navigate around branches to harvest the apples. To remain autonomous the robot should be able to determine its own path without any assistance or supervision.

The current problem is that it is very difficult distinguishing between the branches, leaves and apples. Branches or apples could be partially hidden by many leaves making it very difficult for the robot to distinguish. This is very important as any undetected branches could result in a collision between the robot and a branch. This increases cost as well as maintenance time which significantly undermines the original purpose of the robot. It should be able to very accurately determine the branches to avoid this damage.

The computational solution should be able to intake a RGB and depth image, it should then be able to generate a 3D model of the tree's branch structure, this should be able to be done on the spot to avoid any errors from positioning affecting the result. This should work reliably under many different conditions, the first of which is under heavy occlusion from leaves. Since the robot must be able to pick up apples from the branches it should work well under all of the conditions that would be expected at this time. It should also be able to work well under different environmental conditions, not under perfect lighting.

The scope of this project is to accurately determine the branch structure from the images and depth maps. It will not detect in real time and would be expected to remain still while computation occurs. The branches will be modelled using simplified geometry rather than the exact shape which may not be known. Since the main problem addressed is damage to the robot, as such the project will aim to reduce this, detecting objects such as apples will not be included in the scope. Determining kinematics and calculations on how the robot will move is also out of scope. The biggest limitation is that this gives the robot a way to detect collision avoidance using its own algorithms, rather than avoiding the collision itself. This project is a small subsection of a much larger problem.

The inputs, outputs and data

The final solution requires for a rgb image and the depth map of the apple tree to be input. The RGB image requires a specific size of 640 x 480 px, where the apple tree should be located close to the center of the image. This is required as the image is then center cropped to 480 x 480 px where the tree must remain within this area. This image should contain 3 channels, one for each RGB, these should be formatted in 8-bits where they will then be normalised.

The depth map should be taken at the same location, orientation and time as the RGB image as these should be able to map directly onto each other, this will also be formatted as 640 x 480 px in 8-bit depth.

Unsupported inputs are first filtered, this is because the model does not know how to handle these situations. Invalid data is also filtered out, in row 4 the ground truth masks are all duplicates that do not relate to the RGB or depth images. If any of the data is missing it is also filtered out. If any problems arise the code will raise an error.

The output is a 3D mesh that can be exported as a STL file, this is so it can then be utilised in the algorithm that can determine the pathing of the robot to pick apples, avoiding the branches. This algorithm is out of the scope of this project.

The computational approach

The computational approach involved 2 main stages, the first of which extracted the 2D mask of the tree's underlying branch structure utilising the machine learning model developed. The second of which then took this mask and converted it into a 3D mesh using an algorithm and the depth data. The choice to split up the process into two steps was made to best suit the available data. The dataset contained RGB, depth and ground truth images, these were all 2D. Since the ground truth images were 2D, this made it very easy to validate the performance half way through. Completing everything all in one step would make the problem very difficult to validate, and find the source of errors all at once. A limitation of this process is that errors from the 2D extraction will propagate through the 3D reconstruction formula.

To extract the 2D mask, a neural network following the HRNet structure was developed. This required inputs of the RGB and depth data. A neural network was chosen as there is an enormous amount of settings and parameters that would need to be manually introduced to determine what is and is not a branch. Using a neural network it can be given all of these parameters or weights and learn how to update each one of them to achieve the desired results. The HRNet structure was chosen after many trials with different architectures such as UNet, and it was determined that a HRNet structure was learning and providing the greatest detail.

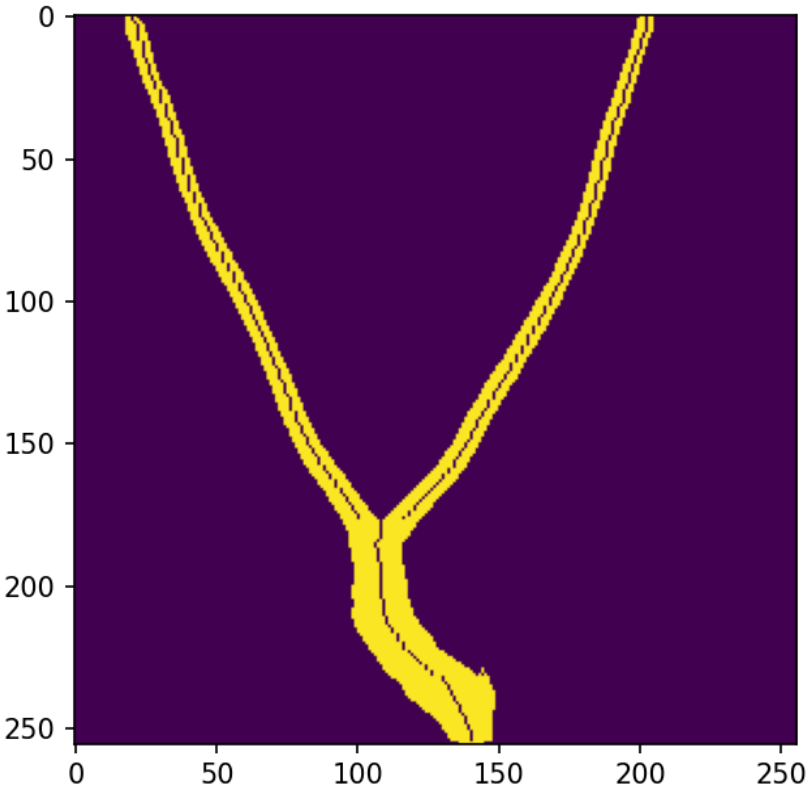
Image augmentation was also used to reduce overfitting, the types of augmentation that were used include: brightness; contrast; shifts; rotations; and flips. Each of these augmentations effectively increases the total amount of training data available, making it harder for the network to memorise images. After implementing this to the model there was a significant improvement in the model's learning. The chosen data split was 80-20, this provided a strong balance between having enough training data as well as having enough validation to be sure. This was done through a randomised split.

The chosen method to characterise the models performance was the binary dice coefficient:

$$Dice = \frac{2|A \cap B|}{|A|+|B|}$$

The output is set as either part of the branch or not part of the branch with a 0 or a 1, this can be done by having a threshold confidence level that the pixel must exceed to be selected as part of the branch, this is the binary step. The dice coefficient is calculated by taking the amount of overlapping area between the predicted output and the ground truth mask, it then doubles this value and divides by the sum of the individual areas. This gives a coefficient between 0 and 1, a value of 0 indicates that the predicted output does not coincide with the ground truth mask at all, while a value of 1 indicates that the prediction is perfect, coinciding with every point of the ground truth and nothing else.

The dice coefficient was selected over other methods like binary-cross-entropy as these can give high scores even if the model is performing poorly. This is because the majority of the background is not a branch, so if a model was to predict nothing as a branch, most of the predicted pixels would be correct anyway.



The next step involved using the 2D mask to convert it to a 3D mesh. The way I chose to overlap this was to take vertical slices of the tree branches, from these slices I could determine the center of each part of the branch as well as the radius of the branch at that point. The depth is then determined through the depth map. It works by fitting a continuous curve through a few points that are predicted to be on the branch. It was also ensured that end conditions are fixed on the branches and trunk to ensure that they meet up. The reason a continuous curve and end conditions were implemented was the physical structure of an apple tree, it is not possible for there to be any large discontinuities in the structure as branches must grow from its current position.

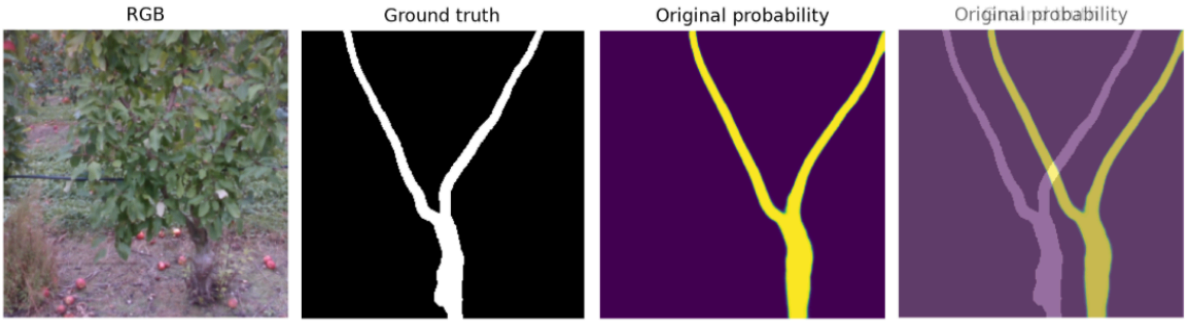
This is the greatest limitation in the entire process, it required many assumptions including how depth mapped to the pixel coordinates and real depth, it was improved until the point where there was no accurate way to verify any further improvements. This happened once the results seemed plausible, with no additional pictures or scans of the actual tree's shape, the depth was difficult to validate. The assumption that was used involved a linear relationship between 0 - 4 meters. The relationship between depth and pixels was also not discussed so this was calculated using the average sizes of apple trees.

The final step involved generating the 3D mesh using these values. Every 5 or so points would be connected by cylinders. As the points were not sorted into branches yet, the branch shape was reconstructed using a minimum spanning tree algorithm. This could then be used to generate the cylinders that form the mesh. A problem with this implementation would result from having branches overlap each other. This current minimum spanning tree algorithm would treat branches that overlap as being connected which could cause some issues.

The validation method and results

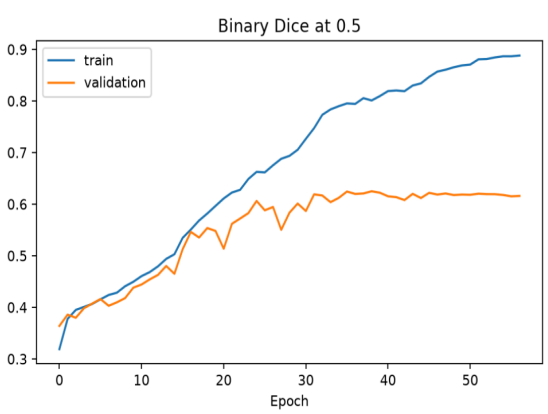
One problem with the dataset and GT that became apparent during the training and trials was that the ground truth masks did not always directly coincide with the center cropped image of the tree. This meant that even if the shape of the mask was a direct match the overlapping area could be very small, since the loss metric dice relies on this, it made it very difficult for the model to improve.

Example of where aligning is needed.

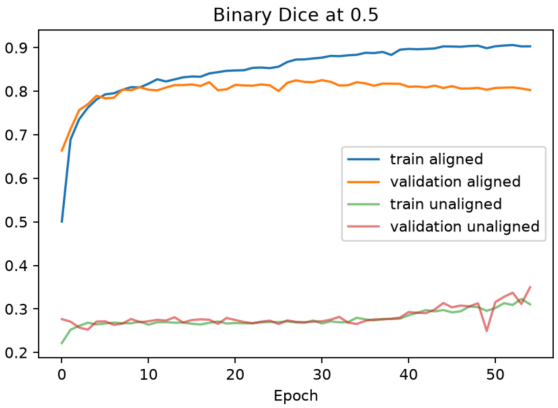


The binary dice loss metric would penalise the model and give it low scores even if it was recognising the correct features and shapes. This made it very easy to overfit the data, as seen in the figure above, validation struggled to achieve a metric above ~ 0.6, this is expected because of the alignment issues, however the training would continue to improve while the validation did not, if the only issue was alignment, then the training should have remained similar to the validation, this signified that the model was overfitting.

The strategy implemented to improve this was by first aligning the predicted model with the ground truth mask and then using binary dice on this. To align the mask a maximum vertical and horizontal shift is predefined, after predicting the shape, it then calculates the dice coefficient after aligning the centroid of the two images. This only translates the predicted up/down and left/right, without distorting the shape in any way, this ensures that the model can't significantly improve the coefficient of any model that is not accurately predicting the shape.



No Auto Aligning



Auto Aligning

In the figure above, it can be seen that the validation has greatly improved, to a coefficient of around 0.8. This indicates a much closer predicted shape. Overfitting can still be seen to occur as the training continues to improve far past the validation curve, the model was stopped early once no clear improvement was to be made. The chosen model weights were from before the model started to overfit, being around epoch 10-15.

The comparison and optimisation work

Optimisation of the Training:

The main bottleneck during model training was the computational cost of finding the correct shift of the predicted masks to align with the target masks. The original method involved testing multiple predefined shifts and choosing the best option out of examples. To improve training efficiency, I tested another model that could accurately predict the middle of each mask and find the required shift. The following involved randomly applied shifts and the results given by the centroid and shifted result.

```
Runtime - 20 trials each
Method                Mean ms  Median ms  Best ms  ms/case
--------------------  -------  ---------  -------  -------
Probability centroid  2.163    2.144      2.113    0.1072
Predefined shifts     57.999   57.423     55.501   2.8712


Testing the two methods

Mask                        applied (x,y)  Centroid Result    Shifted Result
--------------------------  -------------  -----------------  -------------
R14N19_Summer_GT-13-59-55   (8, 2)         (-8.43, -0.65)     (-8, -2)
R14N19_Summer_GT-13-59-55   (-2, -8)       (2.10, 5.15)       (2, 8)
R14N19_Summer_GT-13-59-55   (-2, 0)        (2.00, 0.00)       (2, 0)
R14N19_Summer_GT-13-59-55   (2, -2)        (-1.98, 1.21)      (-2, 2)
R08N07_Summer_GT-13-26-33   (4, 8)         (-4.09, -4.37)     (-4, -8)
R08N07_Summer_GT-13-26-33   (-12, 0)       (12.00, 0.00)      (12, 0)
R08N07_Summer_GT-13-26-33   (-6, 2)        (5.98, -1.12)      (6, -2)
R08N07_Summer_GT-13-26-33   (-4, 6)        (3.94, -3.30)      (4, -6)
R12N20_Summer_GT-13-46-34   (6, -2)        (-5.99, 1.21)      (-6, 2)
R12N20_Summer_GT-13-46-34   (-6, -2)       (6.01, 1.21)       (6, 2)
```

Following this evaluation it was found that the centroid method was 26 times faster than testing all of the predefined shifts. The centroid result was returning near optimal shifts in the horizontal plane while it struggled more with applied vertical shifts. However, since the masks were generated from the same images that were used, and there was no vertical cropping it has been presumed that it is unlikely that there would be any large vertical shifts. Therefore the second method with improved computational efficiency was proven to be effective and replaced the original method.

Optimisation of the 3D Predictions:

The second main area for improvement came in the form of generating the 3D model for the tree. Originally the mesh was created by overlapping many spheres along the branch and then decimating to create an optimised shape. This involved the creation of an enormous amount of faces, edges and vertices which would make calculations and memory use very high. To improve upon this a new method using cylinders was proposed which would significantly reduce the number of faces and edges, making calculations much easier for the apple tree robot. This algorithm reduced the time from 1.1269 s to 0.6735s, a 41% reduction in the algorithm runtime, noting that 0.3 seconds are taken up by the HRNet model. The memory usage also significantly reduced from a peak of 143.86 MB to 47.06 MB, a 67% improvement. The results were nearly identical and factoring in a tolerance zone around the predicted branches meant there would be almost no loss of accuracy as a trade off.

Cylinder Method:

```
Complete pipeline timings:
Run 1: 0.7461 seconds
Run 2: 0.6689 seconds
Run 3: 0.6637 seconds
Run 4: 0.6463 seconds
Run 5: 0.6425 seconds

Minimum: 0.6425 seconds
Median:  0.6637 seconds
Mean:    0.6735 seconds
SD:      0.0421 seconds
```

```
Memory before:          776.46 MB
Peak process memory:    823.53 MB
Peak memory increase:   47.06 MB
Final memory increase:  42.40 MB
```



Sphere Method:

```
Complete pipeline timings:
Run 1: 1.2123 seconds
Run 2: 1.1580 seconds
Run 3: 1.0945 seconds
Run 4: 1.0962 seconds
Run 5: 1.0735 seconds

Minimum: 1.0735 seconds
Median:  1.0962 seconds
Mean:    1.1269 seconds
SD:      0.0572 seconds
```

```
Memory before:          743.74 MB
Peak process memory:    887.60 MB
Peak memory increase:   143.86 MB
Final memory increase:  34.88 MB
```



The limitations and lessons learned

The limitations of each individual step were mentioned in the computational approach. This section will be going over the limitations of the final process as a whole.

The solution accuracy depends very strongly on some of the variables that have not been properly defined, the main example of this is determining the depth where it needs to be to the depth readings with a real world value. An approximation was used and documented but more information about the depth readings would be able to give a better approximation of the depth. The depth of the solution was also not able to be validated past a baseline understanding, as there was no way to validate the depth was correct.

The computational runtime has been reduced to 0.67 seconds, while a large improvement has been made in this area it may still require a faster operation time for more consistent or quicker motion. The memory use was greatly reduced as well to 47.06 MB, a 67% decrease in usage. However, this may still be a very large amount for a small microcontroller or system computer that would be expected to find in a mobile robot.

While brightness and contrast augmentations were used in the training of the neural network to develop the 2D mask, the final output has not been evaluated under real conditions and may fail to accurately predict under these conditions.

The input resolution is scaled down by 640 by 480 px to 256 by 256 px, which means that small details can be lost in this transition. This was not considered to be an issue as the code was designed to obtain the main large branch structure, as small branches would not pose as much of a threat under a collision.

The code has also not been designed to be scalable as it has been designed to take an image of only the tree that it is about to pick an apple from rather than processing many different trees at once.

Conclusion

The final product could produce a 3D model of the main branch structure from just the RGB and depth images. This was done in two stages, the first being a HRNet neural network to predict the 2D mask which was then used to construct the 3D image using the depth data. After improving the HRNet model through image augmentation and aligned masks, the model was able to much more accurately predict the truth, with an aligned dice coefficient of 0.8. The switch to Cylinder based meshes significantly improved performance reducing it by 40% and the peak memory usage down by 67% making it more suitable for remote use. These results could be further improved with the known calibration of the depth sensor as well as known distances from the trees. This project has worked as a good proof of concept, but would still benefit from more testing in unknown conditions such as different lighting, as well as having more images from different angles to help verify the reconstruction accuracy.

AI-use reflection

AI has been used in the assistance of this assessment following all guidelines. The AI used was the free edition of ChatGPT. AI was not used in the development of ideas or methods in which to produce the final output, it was only used as a tool to apply my own ideas.

ChatGPT was used for learning the syntax and giving examples of new modules and libraries that I had not used before such as TensorFlow and TriMesh, it also assisted in the generation of docstrings for functions with the information that I had given it. ChatGPT was also used in the generation of code in some of the files, it only followed instructions and the code was tested with some known test cases to ensure that it would complete the intended task in the intended way. The main example of this was the architecture for the neural network, this could be verified by reading through it line by line, it is difficult to write in a syntax that you do not know well, but reading it is much easier and I could verify that it had created the architecture that I had asked it to use. Other examples could be tested in a few lines such as generating and combining meshes in TriMesh.

This was used to speed up the process of creating the code when I already had an idea and plan that was ready to execute but I could not find many good examples of documentation. There are many risks when involving AI in writing code such as losing a deep understanding of the processes, however it did not affect my understanding as it only followed the instructions that I had told it to do and freed up time for me to focus on other areas. The use of AI was incredibly helpful and sped up the development of the project significantly.

Sources

Keenan Greenland's Apple tree database with ground truth masks.

https://arxiv.org/pdf/2010.06879